Final Project Report

Emily Dunham & Cooper Maughan

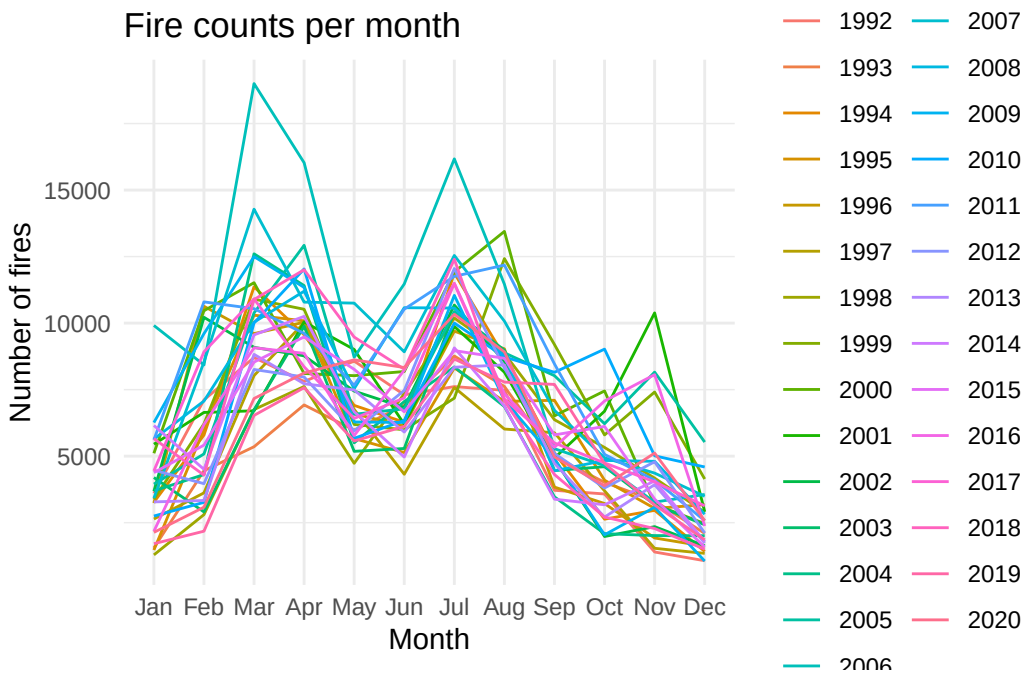
Data Analysis

```
library(ggplot2)
library(tidyverse)
library(refund)
library(gridExtra)

fires <- vroom::vroom("forestfires.csv")
```

Exploratory Analysis

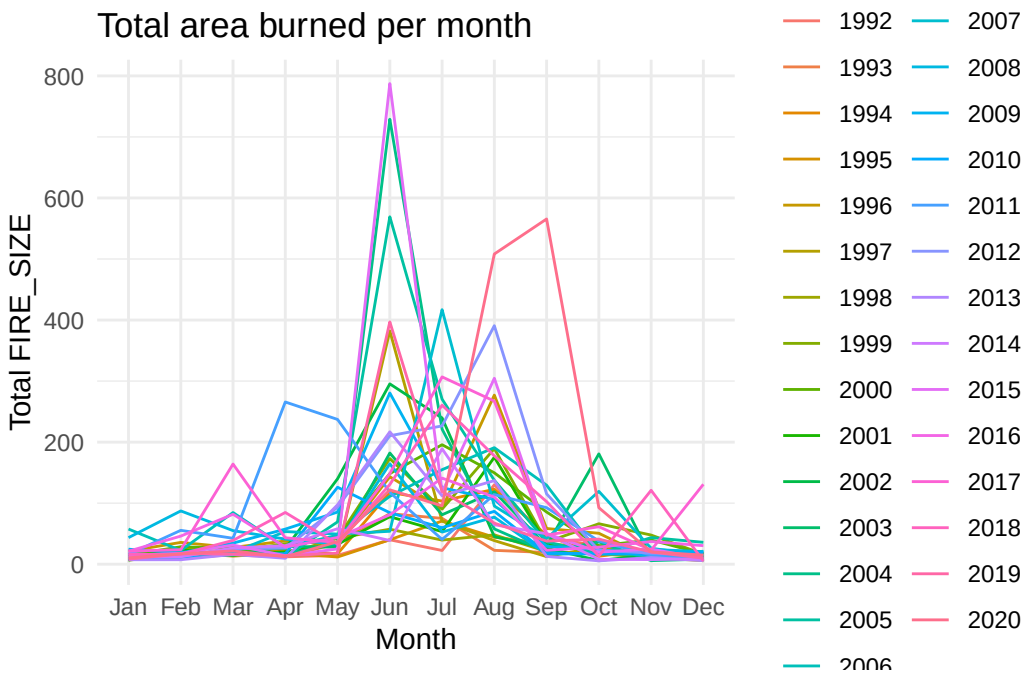
Number of Fires

```
fires_counts <- fires |>
  mutate(
    year  = FIRE_YEAR,
    month = month(DISCOVERY_DATE, label = TRUE)
  ) |>
  count(year, month, name = "n_fires")

ggplot(fires_counts,
       aes(x = month, y = n_fires, color = factor(year), group = year)) +
  geom_line() +
  labs(title = "Fire counts per month",
       x = "Month", y = "Number of fires", color = "Year") +
  theme_minimal()
```



Area Burned

```
fires_month <- fires |>
  mutate(
    year = FIRE_YEAR,
    month = month(DISCOVERY_DATE, label = TRUE)
  ) |>
  group_by(year, month) |>
  summarise(total_size = mean(FIRE_SIZE, na.rm = TRUE), .groups = "drop")

ggplot(fires_month,
       aes(x = month, y = total_size, color = factor(year), group = year)) +
  geom_line() +
  labs(title = "Total area burned per month",
       x = "Month", y = "Total FIRE_SIZE", color = "Year") +
  theme_minimal()
```



Modeling

```
# Making the dataset for our Y matrix
# Taking the values from month column and makes them each their own column
Y_dat <- fires_month |>
  pivot_wider(
    names_from = month,
    values_from = total_size
  )

# Then I want to make this a matrix without the year variable
Y_mat <- as.matrix(Y_dat)[,-1]

# Then do the same for the X matrix
# Create the dataset
X_dat <- fires_counts |>
  pivot_wider(
    names_from = month,
    values_from =n_fires
  )
```

```
# Then I want to make this a matrix
X_mat <- as.matrix(X_dat)[,-1]

t <- 1:ncol(Y_mat)  # response index (months)
s <- 1:ncol(X_mat)  # predictor index (months)

# For total area burned take the log
Y_log <- log(Y_mat+1)

# For fire counts take the log
X_log <- log(X_mat+1)
```

Regular Model

```
FullModel <- pffr(
  Y_log ~ ff(X_log,
             xind = s),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2,1))
)
```

```
Warning in ff(X_log, xind = s): Very low effective rank of <X_log> detected. 1
largest eigenvalues of its covariance alone account for >99.5% of variability.
<ffpc> might be a better choice here.
```

```
Warning in ff(X_log, xind = s): <k> larger than effective rank of <X_log>.
Model identifiable only through penalty.
```

```
FullBetas <- coef(FullModel, n1 = 12, n2 = 12)$smterms[[2]]$value
```

```
using seWithMean for  s(t.vec) .
```

```
R2_reg <- summary(FullModel)
# R-squared of .568
```

Historical Model

```
FullModel_Hist <- pffr(
  Y_log ~ ff(X_log,
             xind = s,
             limits = 's<t'),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2,1))
)
```

```
<limits>-argument detected. Changing to Riemann sums for numerical integration.
```

```
Warning in ff(X_log, xind = s, limits = "s<t"): Very low effective rank of
<X_log> detected. 1 largest eigenvalues of its covariance alone account for
>99.5% of variability. <ffpc> might be a better choice here.
```

```
Warning in ff(X_log, xind = s, limits = "s<t"): <k> larger than effective rank
of <X_log>. Model identifiable only through penalty.
```

```
FullHistBetas <- coef(FullModel_Hist, n1 = 12, n2 = 12)$smterms[[2]]$value
```

```
using seWithMean for  s(t.vec) .
```

```
R2_hist <- summary(FullModel_Hist)
# R-squared of .6
```

Plotting the Models

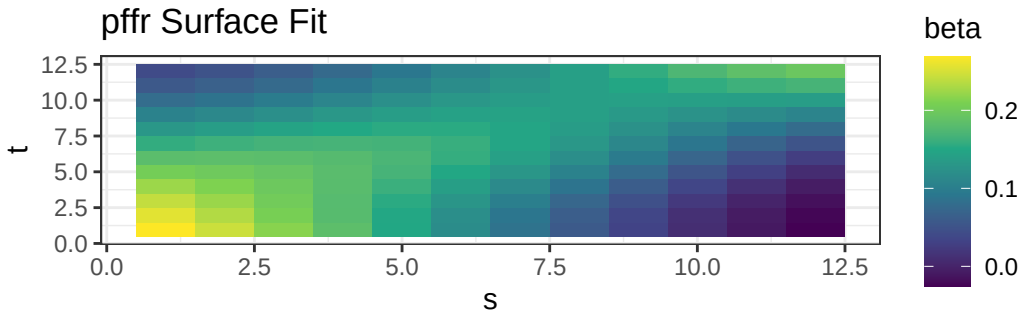
```
FullModel2 <- coef(FullModel, n1 = 12, n2 = 12)$smterms[[2]]$coef
```

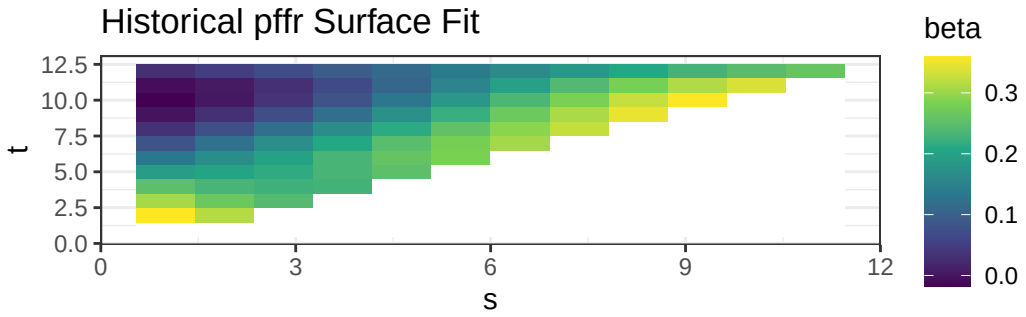
```
using seWithMean for  s(t.vec) .
```

```
FullModel2_Hist <- coef(FullModel_Hist, n1 = 12, n2 = 12)$smterms[[2]]$coef
```

```
using seWithMean for  s(t.vec) .
```

```r
# Creating the dataframe
FullBetasdf <- data.frame(
  't'    = c(FullModel2$X_log.tmat, FullModel2_Hist$X_log.tmat),
  's'    = c(FullModel2$X_log.smat, FullModel2_Hist$X_log.smat),
  'beta' = c(FullModel2$value, FullModel2_Hist$value),
  'Fit'  = factor(rep(c('Full pffr', 'Historical pffr'),
                      times = c(nrow(FullModel2),
                                nrow(FullModel2_Hist))))
)

# Separating the dataframes into two
# Regular
Full <- ggplot(data = FullBetasdf[FullBetasdf$Fit == "Full pffr", ]) +
  geom_raster(aes(x = s, y = t, fill = beta)) +
  scale_fill_viridis_c() +
  xlab('s') + ylab('t') + theme_bw() +
  ggtitle('pffr Surface Fit')

# Historical
HistPlotData <- FullBetasdf[FullBetasdf$Fit == "Historical pffr", ]

# Set beta to NA wherever s >= t
HistPlotData$beta[HistPlotData$s >= HistPlotData$t] <- NA

Full_Hist <- ggplot(data = HistPlotData) +
  geom_raster(aes(x = s, y = t, fill = beta)) +
  scale_fill_viridis_c(na.value = "white") +
  xlab('s') + ylab('t') + theme_bw() +
  ggtitle('Historical pffr Surface Fit')


grid.arrange(Full, Full_Hist)
```





Predictions

```
area_train <- fires_month |>
  filter(year <= 2015)
area_test <- fires_month |>
  filter(year >= 2016)

fires_train <- fires_counts |>
  filter(year <= 2015)
fires_test <- fires_counts |>
  filter(year >= 2016)

## Y Matricies
# Make the training dataset a matrix for the model
Y_dat_area_train <- area_train |>
  pivot_wider(
    names_from = month,
    values_from = total_size
  )
Y_area_train <- as.matrix(Y_dat_area_train)[,-1]
```

```r
# Make the test dataset a matrix
Y_dat_area_test <- area_test |>
  pivot_wider(
    names_from = month,
    values_from = total_size
  )
Y_area_test <- as.matrix(Y_dat_area_test)[,-1]

## X matricies
# Make the training dataset a matrix for the model
X_dat_fires_train <- fires_train |>
  pivot_wider(
    names_from = month,
    values_from =n_fires
  )
X_fires_train <- as.matrix(X_dat_fires_train)[,-1]

# Make the test dataset a matrix
X_dat_fires_test <- fires_test |>
  pivot_wider(
    names_from = month,
    values_from =n_fires
  )
X_fires_test <- as.matrix(X_dat_fires_test)[,-1]

t <- 1:ncol(Y_area_train)  # response index (months)
s <- 1:ncol(X_fires_train)  # predictor index (months)

# For total area burned take the log
Y_log_train <- log(Y_area_train+1)

# For fire counts take the log
X_log_train <- log(X_fires_train+1)
```

Full Train Model

```r
Train_Model <- pffr(
  Y_log_train ~ ff(X_log_train,
             xind = s),
  yind = t,
```

```
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2,1))
)
```

```
Warning in ff(X_log_train, xind = s): Very low effective rank of <X_log_train>
detected. 1 largest eigenvalues of its covariance alone account for >99.5% of
variability. <ffpc> might be a better choice here.
```

```
Warning in ff(X_log_train, xind = s): <k> larger than effective rank of
<X_log_train>. Model identifiable only through penalty.
```

```
Train_FullBetas <- coef(Train_Model, n1 = 12, n2 = 12)$smterms[[2]]$value
```

```
using seWithMean for  s(t.vec) .
```

```
R2_full_train <- summary(Train_Model)
# R^2 = 0.613
```

Historical Train Model

```
Train_Model_Hist <- pffr(
  Y_log_train ~ ff(X_log_train,
             xind = s,
             limits = 's<t'),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2,1))
)
```

```
<limits>-argument detected. Changing to Riemann sums for numerical integration.
```

```
Warning in ff(X_log_train, xind = s, limits = "s<t"): Very low effective rank
of <X_log_train> detected. 1 largest eigenvalues of its covariance alone
account for >99.5% of variability. <ffpc> might be a better choice here.
```

```
Warning in ff(X_log_train, xind = s, limits = "s<t"): <k> larger than effective
rank of <X_log_train>. Model identifiable only through penalty.
```

```
Train_HistBetas <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$smterms[[2]]$value
```

```
using seWithMean for  s(t.vec) .
```

```
R2_hist_train <- summary(Train_Model_Hist)
# R-squared of .637
```

Regular Model Prediction

```
X_log_test <- log(X_fires_test +1)
reg.train_preds <- predict(Train_Model, newdata = list(X_log_train = X_log_test))

# Look at RMSE
y_test <- t(Y_area_test)
reg.train_preds_t <- t(reg.train_preds)

regular_rmse <- sqrt(mean((y_test - reg.train_preds_t)^2))
regular_rmse
```

```
[1] 141.591
```

Historical Model Prediction

```
## We need to do this manually
## Predict function doesn't work with limits

# Get intercept
smterms <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$smterms[[1]]$value
```

```
using seWithMean for  s(t.vec) .
```

```
pterms <- coef(Train_Model_Hist, n1 = 12, n2 = 12)$pterms[1,1]
```

```
using seWithMean for  s(t.vec) .
```

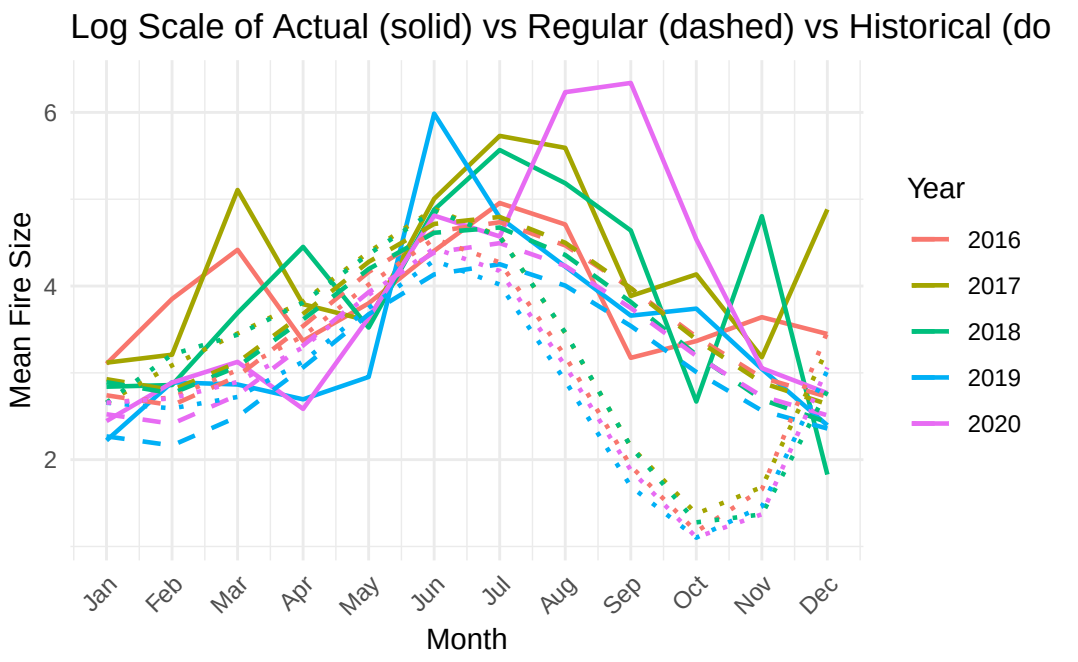
```r
b0 <- pterms + smterms
b0_matrix <- matrix(b0, nrow = 12, ncol = 5)

# Get the beta surface matrix (12 x 12)
b1 <- Train_HistBetas
b1 <- matrix(b1, nrow = 12, ncol = 12)
b1[lower.tri(b1, diag = TRUE)] <- 0 # taking out values where s>=t

# Manual prediction
b1_matrix <- X_log_test %*% b1
b1_matrix_t <- t(b1_matrix)

hist.train_preds <- b0_matrix + b1_matrix_t

## Get RMSE's
historical_rmse <- sqrt(mean((y_test - hist.train_preds)^2))
historical_rmse
```

```
[1] 141.9687
```

Plot Predictions (log scale)

```r
test_years <- 2016:2020
Y_log_test <- log(Y_area_test +1)

# --- Actual values (long format)
actual_long <- as.data.frame(Y_log_test) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "actual") |>
  mutate(month = as.numeric(month))

# Then redo the long format with back-transformed values
reg_long <- as.data.frame(reg.train_preds) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "pred_reg") |>
  mutate(month = as.numeric(month))

hist_long <- as.data.frame(t(hist.train_preds)) |>
```

```r
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "pred_hist") |>
  mutate(month = as.numeric(month))

# Re-join and replot
plot_dat <- actual_long |>
  left_join(reg_long, by = c("year", "month")) |>
  left_join(hist_long, by = c("year", "month")) |>
  mutate(year = factor(year))

ggplot(plot_dat, aes(x = month, color = year, group = year)) +
  geom_line(aes(y = actual), linewidth = 0.8) +
  geom_line(aes(y = pred_reg), linetype = "dashed", linewidth = 0.8) +
  geom_line(aes(y = pred_hist), linetype = "dotted", linewidth = 0.8) +
  scale_x_continuous(breaks = 1:12, labels = month.abb) +
  labs(
    title = "Log Scale of Actual (solid) vs Regular (dashed) vs Historical (dotted) Predictio
    x = "Month", y = "Mean Fire Size", color = "Year"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Plot Predictions

```
Train_Model_nl <- pffr(
  Y_area_train ~ ff(X_fires_train,
                    xind = s),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2,1))
)

Train_FullBetas_nl <- coef(Train_Model_nl, n1 = 12, n2 = 12)$smterms[[2]]$value
```

```
using seWithMean for  s(t.vec) .
```

```
R2_full_train_nl <- summary(Train_Model_nl)
# R^2 = 0.292

Train_Model_Hist_nl <- pffr(
  Y_area_train ~ ff(X_fires_train,
                    xind = s,
                    limits = 's<t'),
  yind = t,
  method = "NCV",
  bs.int = list(bs = "ps", k = 5, m = c(2,1))
)
```

```
<limits>-argument detected. Changing to Riemann sums for numerical integration.
```

```
Train_HistBetas_nl <- coef(Train_Model_Hist_nl, n1 = 12, n2 = 12)$smterms[[2]]$value
```

```
using seWithMean for  s(t.vec) .
```

```
R2_hist_train_nl <- summary(Train_Model_Hist_nl)
# R-squared of .321

reg.train_preds_nl <- predict(Train_Model_nl, newdata = list(X_fires_train = X_fires_test))

# Look at RMSE
y_test <- t(Y_area_test)
```

```r
reg.train_preds_t_nl <- t(reg.train_preds_nl)

regular_rmse_nl <- sqrt(mean((y_test - reg.train_preds_t_nl)^2))
regular_rmse_nl
```

```
[1] 104.2018
```

```r
#104.20

# Get intercept
smterms <- coef(Train_Model_Hist_nl, n1 = 12, n2 = 12)$smterms[[1]]$value
```

```
using seWithMean for  s(t.vec) .
```

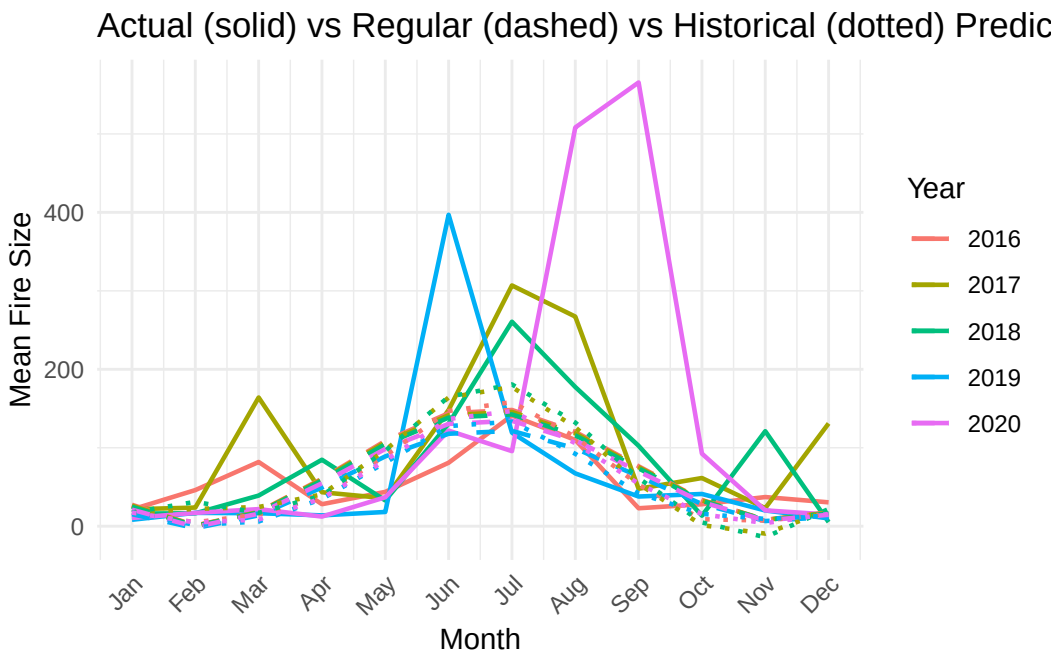
```r
pterms <- coef(Train_Model_Hist_nl, n1 = 12, n2 = 12)$pterms[1,1]
```

```
using seWithMean for  s(t.vec) .
```

```r
b0 <- pterms + smterms
b0_matrix <- matrix(b0, nrow = 12, ncol = 5)

# Get the beta surface matrix (12 x 12)
b1 <- Train_HistBetas_nl
b1 <- matrix(b1, nrow = 12, ncol = 12)
b1[lower.tri(b1, diag = TRUE)] <- 0 # taking out values where s>=t

# Manual prediction
b1_matrix <- X_fires_test %*% b1
b1_matrix_t <- t(b1_matrix)

hist.train_preds_nl <- b0_matrix + b1_matrix_t

## Get RMSE's
historical_rmse_nl <- sqrt(mean((y_test - hist.train_preds_nl)^2))
historical_rmse_nl
```

```
[1] 103.5846
```

```r
# 103.58

test_years <- 2016:2020

# --- Actual values (long format)
actual_long <- as.data.frame(Y_area_test) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "actual") |>
  mutate(month = as.numeric(month))

# Then redo the long format with back-transformed values
reg_long <- as.data.frame(reg.train_preds_nl) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "pred_reg") |>
  mutate(month = as.numeric(month))

hist_long <- as.data.frame(t(hist.train_preds_nl)) |>
  setNames(1:12) |>
  mutate(year = test_years) |>
  pivot_longer(-year, names_to = "month", values_to = "pred_hist") |>
  mutate(month = as.numeric(month))

# Re-join and replot
plot_dat <- actual_long |>
  left_join(reg_long, by = c("year", "month")) |>
  left_join(hist_long, by = c("year", "month")) |>
  mutate(year = factor(year))

ggplot(plot_dat, aes(x = month, color = year, group = year)) +
  geom_line(aes(y = actual), linewidth = 0.8) +
  geom_line(aes(y = pred_reg), linetype = "dashed", linewidth = 0.8) +
  geom_line(aes(y = pred_hist), linetype = "dotted", linewidth = 0.8) +
  scale_x_continuous(breaks = 1:12, labels = month.abb) +
  labs(
    title = "Actual (solid) vs Regular (dashed) vs Historical (dotted) Predictions",
    x = "Month", y = "Mean Fire Size", color = "Year"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```


Actual (solid) vs Regular (dashed) vs Historical (dotted) Predic
Mean Fire Size
400
200
0
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Month
Year
2016
2017
2018
2019
2020